



Computer Engineering Department

Course name: Web Technologies

Course code: 2CP08

Faculty: Prof Kirtikumar Sharma

ID: 23CP029

Name: Viral Vipulkumar Bhoi

Topic: Report of LAB - 7

LAB 7: JavaScript:2 (Events and HTML Document Object Model)

TABLE OF CONTENT:

7.1 Brief introduction of used JS	3
7.2 Exercise 1	4
7.3 Exercise 2	6
7.4 Exercise 3	7
7.5 Exercise 4	9
7.6 Exercise 5	10
7.7 Exercise 6	11
7.8 Exercise 7	13
7.9 Exercise 8	15
7.10 Exercise 9	18
7.11 Exercise 10	23

7.1 Brief introduction of used JS::

1. navigator Object:

- Gives information about the browser (name, version, OS, etc.).
- e.g. `console.log(navigator.userAgent);`

2. history Object:

- Helps navigate through browsing history.
- `history.back();` // Go to the previous page
- `history.forward();` // Go to the next page

3. location Object:

- Gives URL details and allows page redirection.
- `console.log(location.href);` // Shows the current page URL
- `location.href = "https://google.com";` // Redirect to Google

4. event Object:

- Stores details of user actions like clicks, keypresses, etc.
- `document.addEventListener("click", (event) => {
 console.log("Clicked at: X =", event.clientX, "Y =", event.clientY);
});`

5. Document Object Model (DOM):

- **The Document Object Model (DOM)** is a tree-like structure that represents a web page. It allows JavaScript to access and modify HTML and CSS dynamically.
- The browser converts an **HTML page** into a **DOM tree**.
- JavaScript can **add, change, or remove** elements in the DOM.
- The DOM has a **hierarchical structure**

6. Accessing Elements in the DOM:

- `getElementById`:
 - Selects an element using its id.
 - `document.getElementById("myId").innerText = "Hello!"`;
- `getElementsByName`
 - Selects elements with the same name attribute.
 - `let inputs = document.getElementsByName("username");`
- `getElementsByClassName`
 - Selects elements by class
 - `let items = document.getElementsByClassName("myClass");`

7. addEventListener:

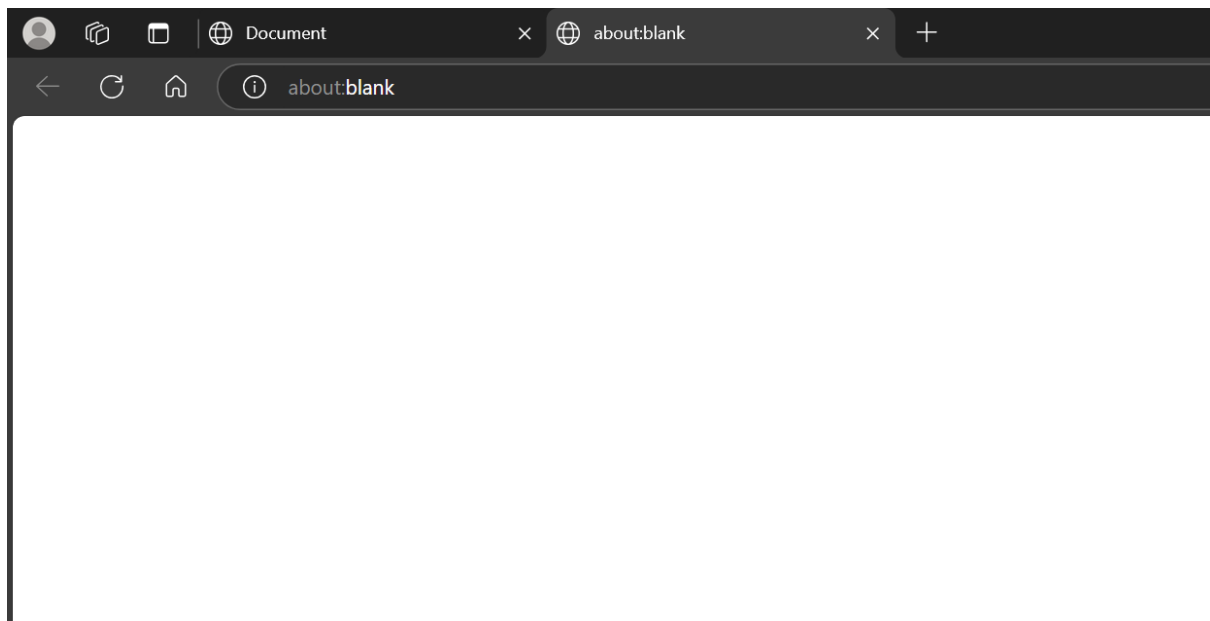
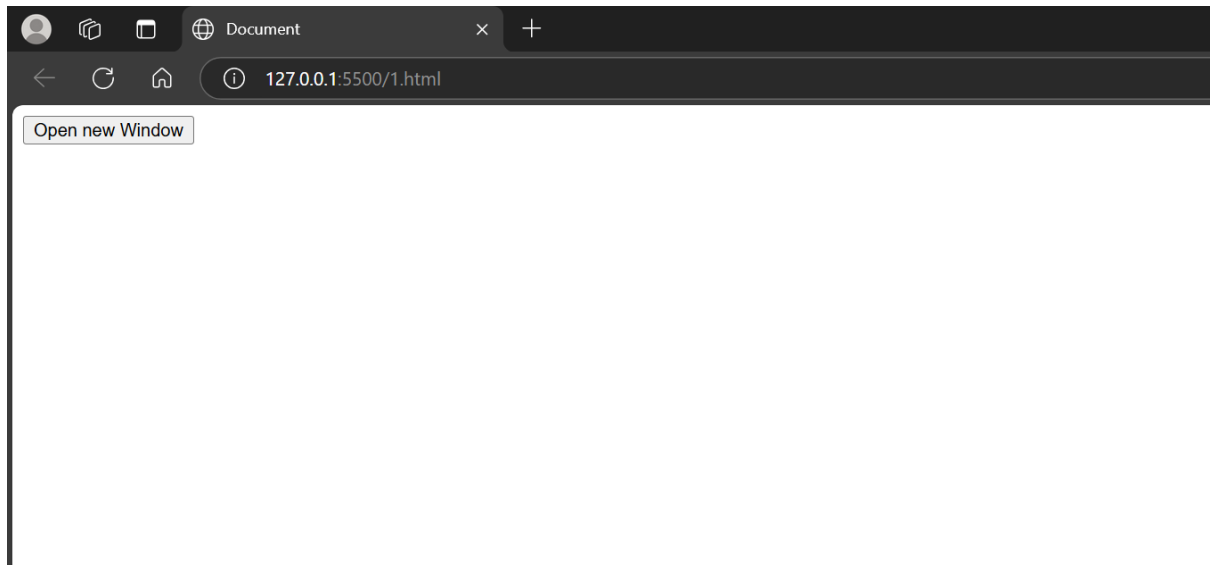
- Adds an event to an element.
- `document.getElementById("btn").addEventListener("click", () => {
 alert("Button clicked!");
});`

7.2 Exercise 1: Write a JavaScript to create a button. On click on the button a new window should open up

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <button onclick="openWindow()">Open new Window</button>
  <script>
    function openWindow(){
      window.open();
    }
  </script>
</body>
</html>
```

OUTPUT:



7.3 Exercise 2: Write a JavaScript to get details of location object.

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    for(i in location){
      document.writeln(i, " : ",location[i]);
      document.writeln("<br>");
    }
  </script>
</body>
</html>
```

OUTPUT:

```
ancestorOrigins : [object DOMStringList]  
href : http://127.0.0.1:5500/2.html  
origin : http://127.0.0.1:5500  
protocol : http:  
host : 127.0.0.1:5500  
hostname : 127.0.0.1  
port : 5500  
pathname : /2.html  
search :  
hash :  
assign : function assign() { [native code] }  
reload : function reload() { [native code] }  
replace : function replace() { [native code] }  
toString : function toString() { [native code] }
```

7.4 Exercise 3: Write a JavaScript to get details of navigator object .

HTML:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
</head>  
<body>
```

```

<script>

    for(i in navigator){

        document.writeln(i," : ",navigator[i]);

        document.writeln("<br>");

    }

</script>

</body>

</html>

```

OUTPUT:

```

vendorSub :
productSub : 20030107
vendor : Google Inc.
maxTouchPoints : 10
scheduling : [object Scheduling]
userActivation : [object UserActivation]
doNotTrack : null
geolocation : [object Geolocation]
connection : [object NetworkInformation]
plugins : [object PluginArray]
mimeTypes : [object MimeTypeArray]
pdfViewerEnabled : true
webkitTemporaryStorage : [object DeprecatedStorageQuota]
webkitPersistentStorage : [object DeprecatedStorageQuota]
windowControlsOverlay : [object WindowControlsOverlay]
hardwareConcurrency : 8
cookieEnabled : true
appName : Mozilla
appVersion : 5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36 Edg/133.0.0.0
platform : Win32
product : Gecko
userAgent : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36 Edg/133.0.0.0
language : en-US
languages : en-US,en,hi,gu,en-IN
onLine : true
webdriver : false
getGamepads : function getGamepads() { [native code] }
javaEnabled : function javaEnabled() { [native code] }
sendBeacon : function sendBeacon() { [native code] }
vibrate : function vibrate() { [native code] }
deprecatedRunAdAuctionEnforcesKAnonymity : false
protectedAudience : [object ProtectedAudience]
bluetooth : [object Bluetooth]
storageBuckets : [object StorageBucketManager]

```


7.5 Exercise 4: Write a JavaScript to retrieve browser name, according browser name open specific version of index page. i.e. chrome version.html, edge version.html

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    let userAgent = navigator.userAgent.toLowerCase();
    if (userAgent.includes("chrome") && !userAgent.includes("edg")) {
      // window.location.href = "chrome_version.html";

      document.write("CHROME VERSION")
    } else if (userAgent.includes("edg")) {
      // window.location.href = "edge_version.html";
      document.writeln("EDGE VERSION");
    } else {
      alert("Unsupported browser!");
    }
  </script>
</body>
</html>
```

OUTPUT:

EDGE VERSION

7.6 Exercise 5: Write a JavaScript to get details of History object.

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    for(i in history){
      document.writeln(i, " : ",history[i]);
      document.writeln("<br>");
    }
  </script>
</body>
</html>
```

OUTPUT:

```
length : 1
scrollRestoration : auto
state : null
back : function back() { [native code] }
forward : function forward() { [native code] }
go : function go() { [native code] }
pushState : function pushState() { [native code] }
replaceState : function replaceState() { [native code] }
```

7.7 Exercise 6: Write a JavaScript to Work with change event that occurs when a different item in the selection list is selected. Use innerHTML.

HTML:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body onload="func()" onreset="func()">
    <form action="" name="frm">
      <select name="item" id="item" onchange="func()">
        <option value="Monitor">Monitor</option>
        <option value="HDD">HDD</option>
      </select>
```

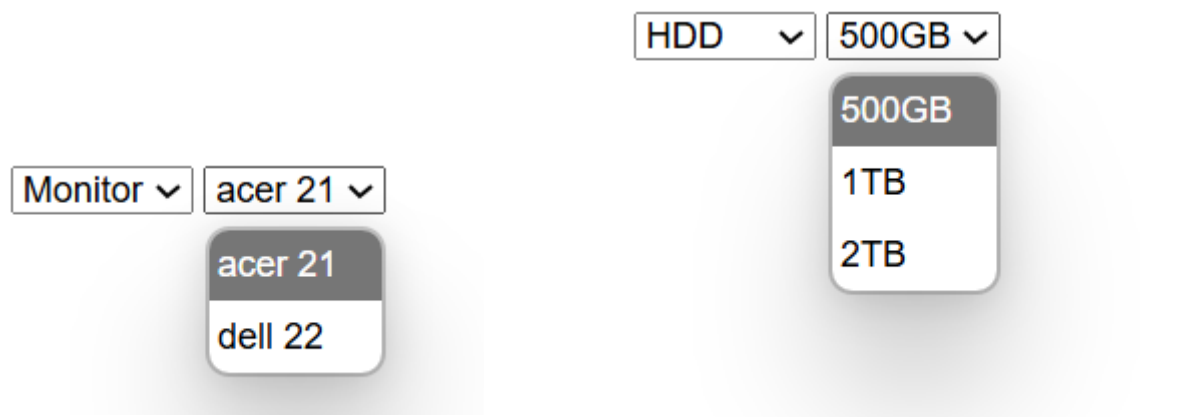
```
<select name="prod" id="prod"></select>
</form>
<script>
function func() {
    let item = {
        Monitor: ["acer 21", "dell 22"],
        HDD: ["500GB", "1TB", "2TB"],
    };
    let option = document.getElementById("item").value;
    let s = "";

    let tempObj = { 0: "Monitor", 1: "HDD" };

    for (i in item[option]) {
        s += "<option>" + item[option][i] + "</option>";
    }

    let change = document.getElementById("prod");
    change.innerHTML = s;
}
// func();
</script>
</body>
</html>
```

OUTPUT:



7.8 Exercise 7: Write a JavaScript to retrieve mouse location and display Unicode character and its key code enter from keyboard using event object.

HTML:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>

    <h2>Mouse and Keyboard Event Tracker</h2>

    <p id="pos">mouse coordinates.</p>
    <p id="key">Pressed key</p>

    <script>
```

```
addEventListener("mousemove", (event) => {  
    document.getElementById("pos").innerText =  
        "Mouse Position - X: " + event.clientX + ", Y: " + event.clientY;  
});  
  
addEventListener("keypress", (event) => {  
    document.getElementById("key").innerText =  
        "Key Pressed: " + event.key + " (Code: " + event.keyCode + ")";  
});  
  
</script>  
</body>  
</html>
```

OUTPUT:

Mouse and Keyboard Event Tracker

Mouse Position - X: 945, Y: 471

Key Pressed: h (Code: 104)

Mouse and Keyboard Event Tracker

Mouse Position - X: 1220, Y: 543

Key Pressed: r (Code: 114)

7.9 Exercise 8: Write a JavaScript to validate registryourself.html page. Give alert message if data is blank. Validate Email, name, password, mobile filed.

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h2>Register Yourself</h2>
  <form onsubmit="return validateForm()">
    <label for="name">Name:</label>
```

```

<input type="text" id="name" name="name"><br><br>
<label for="email">Email:</label>
<input type="email" id="email" name="email"><br><br>
<label for="password">Password:</label>
<input type="password" id="password" name="password"><br><br>
<label for="mobile">Mobile:</label>
<input type="text" id="mobile" name="mobile"><br><br>
<button type="submit">Register</button>
</form>
<script>
function validateForm() {
    let name = document.getElementById("name").value.trim();
    let email = document.getElementById("email").value.trim();
    let password = document.getElementById("password").value.trim();
    let mobile = document.getElementById("mobile").value.trim();
    let error = "";
    for (let i = 0; i < name.length; i++) {
        let char = name[i];
        if (!((char >= 'A' && char <= 'Z') || (char >= 'a' && char <= 'z') || char === ' ')) {
            error += "Name should contain only letters and spaces.\n";
            break;
        }
    }

    if (email.indexOf('@') === -1 || email.indexOf('.') === -1 || email.indexOf('@') >
email.lastIndexOf('.')) {
        error += "Please enter a valid email address.\n";
    }

    if (password.length < 6) {
        error += "Password must be at least 6 characters long.\n";
    }
}

```



```
    }  
    if (mobile.length !== 10) {  
        error += "Mobile number must be exactly 10 digits.\n";  
    }  
    for (let i = 0; i < mobile.length; i++) {  
        if (mobile[i] < '0' || mobile[i] > '9') {  
            error += "Mobile number should contain only digits.\n";  
            break;  
        }  
    }  
    if (error !== "") {  
        alert(error);  
        return false;  
    } else {  
        errorDiv.innerHTML = "";  
        alert("Registration successful!");  
        return true;  
    }  
}  
</script>  
</body>  
</html>
```

OUTPUT:

Register Yourself

Name:

Email:

Password:

Mobile:

127.0.0.1:5500 says

Please enter a valid email address.
Password must be at least 6 characters long.
Mobile number must be exactly 10 digits.

OK

Register Yourself

Name:

Email:

Password:

Mobile:

127.0.0.1:5500 says

Name should contain only letters and spaces.

OK

7.10 Exercise 9: Create a stopwatch.

HTML:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Document</title>
```

```
  <style>
```

```
#container{
  width:100vw;
  height:100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
}
</style>
</head>
<body>
  <div id="container">
    <div id="TIME">
      00 : 00 : 000
    </div>
    <div>
      <button id="Start">START</button>
      <button id="Lap">LAP</button>
      <button id="reset">RESET</button>
      <div id="lapTiming"></div>
    </div>
  </div>

  <script>
    let defTime = "00 : 00 : 000";
    let time = document.getElementById("TIME");
    let startBtn = document.getElementById("Start");
    let lapBtn = document.getElementById("Lap");
    let resetBtn = document.getElementById("reset");
```

```
let lapTime = document.getElementById("lapTiming")  
  
let milisec = 0;  
  
let on = false;  
  
let timingFunc;
```

```
startBtn.addEventListener("click", (event)=> {  
  if(on){  
    on = false;  
    clearInterval(timingFunc);  
    startBtn.innerText = "START";  
    // time.innerText = defTime;  
  }else{  
    timingFunc = setInterval(()=>{  
      milisec++;  
      setTime(milisec);  
    },1);  
    on = true;  
    startBtn.innerText = "STOP";  
  }  
})
```

```
lapBtn.addEventListener("click", (event)=> {  
  lapTime.innerHTML += `<p> ${time.innerText} </p>`;  
})
```

```
resetBtn.addEventListener("click", (event)=> {  
  milisec = 0;  
  on = false;  
  clearInterval(timingFunc);
```

```

startBtn.innerText = "START";
time.innerText = defTime;
lapTime.innerHTML = "";
})

function setTime(milisec){
    let min = Math.floor((milisec/60000) % 60);
    let sec = Math.floor((milisec/1000) % 60);
    let msec = Math.floor(milisec % 1000);
    let min_s,sec_s,msec_s;

    if(min < 10){
        min_s = '0' + min;
    }else{
        min_s = "" + min;
    }

    if(sec < 10){
        sec_s = '0' + sec;
    }else{
        sec_s = "" + sec;
    }

    if(msec < 10){
        msec_s = '00' + msec;
    }else if(msec >= 10 && msec < 100){
        msec_s = "0" + msec;
    }else{
        msec_s = "" + msec;
    }

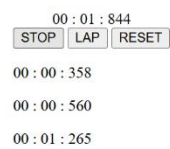
```

```
    }

    time.innerText = `${min_s} : ${sec_s} : ${msec_s}`;
  }

</script>
</body>
</html>
```

OUTPUT:



The screenshot shows a digital timer interface. At the top, the time is displayed as 00:01:844. Below the time display are three buttons: STOP, LAP, and RESET. Below the buttons, there are three lines of text: 00:00:358, 00:00:560, and 00:01:265.

7.11 Exercise 10: if user don't make any activity for 30 second then logout and redirect to logout page.

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1>window will Logged out in 30 sec if no activity by user</h1>
  <p>activity = mousemove, keypress</p>

  <script>
    let timingFunc;

    function resetTimer() {
      clearTimeout(timingFunc);
      timingFunc = setTimeout(() => {
        alert("Logged out because you are inactive");
        window.location.href = "Logout.html";
      }, 30000);
    }

    document.addEventListener("mousemove", resetTimer);
    document.addEventListener("keypress", resetTimer);
```

```
    resetTimer();  
</script>
```

```
</body>
```

```
</html>
```

OUTPUT:

window will Logged in 30 sec

activity = mousemove, keypress

127.0.0.1:5500 says
Logged out because you are inactive

OK